# Ballarch: Boundaries in Time and Honest Outcomes, from Register-Transfer Logic to Services

Vladislav Tomilov
Independent researcher
https://github.com/4wl2d/Pokeball

October 2026
Specification version 2.0.0-rc.1

**Abstract**

Software architecture styles draw boundaries in space: layers, hexagons and services constrain which code may depend on which. They say little about *when* an effect may leave a component, or about what the component may claim afterwards. When a request times out, two-valued error handling has to choose between "it happened" and "it did not". When the destination cannot deduplicate requests, either choice can be wrong: retrying can charge a customer twice, and reporting failure can hide a charge that happened.

We present Ballarch, an architecture that adds two *boundaries in time* to the usual spatial ones. The *commit horizon* is the atomic commit of the decision that requests an effect. The *send horizon* is the first moment the request may have reached its destination. Each piece of mutable state has one owner that decides in pure functions. A runtime commits the decision's state and its requested effects atomically. Every outbound request then ends in exactly one outcome, Done, NotDone or Unknown, chosen mechanically from the request's phase relative to the send horizon and from the effect class of its port. For operations made of several requests, a knowledge algebra over Kleene's strong three-valued logic determines what a component may claim. We show that composing honest parts yields an honest whole, so a composite can be used one level up as a single component.

The rules are stated once and, where they apply, checked by level-specific mechanisms in three references: a Kotlin service runtime with trace oracles and an Alloy model of the call protocol, C firmware with exhaustive power-loss exploration, and a SystemVerilog block with structural netlist checks. In a benchmark of one checkout feature written in six styles and run 2,000 times per fault setting, idiomatic layered, actor, MVU and outbox-plus-saga versions violated a correctness requirement in 12–73% of runs. Ballarch and a layered version hardened by hand violated none. Against the hardened version, Ballarch used about 29% more code and many more framework types, and its classes scored lower on cohesion metrics. It had the lowest propagation cost of the six designs and placed 88% of its code in pure classes.

## 1 Introduction

A payment provider that does not deduplicate requests receives a charge request and charges the card; the answer is then lost. The client times out. What should it record?

- If it retries, the customer may be charged twice.
- If it reports a failure, a charged customer is told that nothing happened.

The only honest answer is a third one: the charge *may* have happened, and someone has to find out. The "third outcome" of remote procedure calls is old knowledge [5]. Distributed-systems testing

records such operations as indeterminate [30, 31], and transaction standards report heuristic and hazard outcomes [56]. Hardware solved a related problem long ago: processors retire instructions in order so that exceptions are precise [52], and memory types that model devices forbid speculative or repeated accesses [1].

Architecture styles for application software do not, however, place these facts in the architecture:

- Layered and Clean Architecture [39], Ports and Adapters [13] and domain-driven design [16] constrain dependencies.
- Actors [2, 10, 26] and the Elm architecture [14] give state a single owner.
- Sagas [19] and the transactional outbox [50] keep state and messages consistent.

None of them says when an effect may leave a component, or what the component may claim after a timeout. Each feature author decides. In our benchmark (Section 8), straightforward implementations in four styles charged customers twice or reported charged orders as failed in 12–73% of faulty runs. A version hardened by an expert was correct, but its correctness lived in the feature code.

This paper presents Ballarch, an architecture whose rules make that knowledge structural. Ballarch keeps established mechanisms (single owners, pure decision functions, commit then dispatch, adapters at the edge) and cites them as such. Its contributions are the following:

1. **Boundaries in time** (Section 3). Every interaction crosses two instants in order: the *commit horizon* and the *send horizon*. They are architectural boundaries alongside spatial ones, and every implementation must make them observable.

2. **Outcomes from phase and effect class** (Section 4). A runtime chooses each request's single outcome mechanically. It never reports NotDone without evidence and never re-sends a non-idempotent request.

3. **A knowledge algebra with composition closure** (Section 5). Knowledge of a composite operation is a pair (*all*, *any*) over Kleene's strong logic, with absorbing compensations. Sound parts compose into a sound and coordinate-wise exact whole, so a composite can be exposed as one honest port.

4. **One rule set, checked at every level** (Section 6). Fourteen rules are stated once. Wherever they apply, they are checked by level-specific mechanisms, in references for services (Kotlin), firmware (C) and synchronous hardware (SystemVerilog).

We evaluate the architecture with an Alloy model of the call protocol, exhaustive checks of the algebra, fault exploration at each level, and a benchmark comparing six architectural styles (Section 8). All artifacts are public [54].

## 2 Model

**Balls and deciders.** A *Ball* is a single-writer owner of state, addressed by a type and a key. Its type is defined by a *decider*, a set of pure functions:

- `decide(state, request, ctx)` returns *Accept*(state′, reply plan, outputs) or *Reject*(reason);
- `complete(state, completion, ctx)` and `observe(state, notice, ctx)` return a step (state′, outputs).

Outputs are *calls* to ports, *replies*, *notices* and *cancellations*. The context supplies time, identity and fresh call keys, so deciders read nothing ambient. The idea of a decider is not new [11, 14]; what is new is what the runtime does with its outputs.

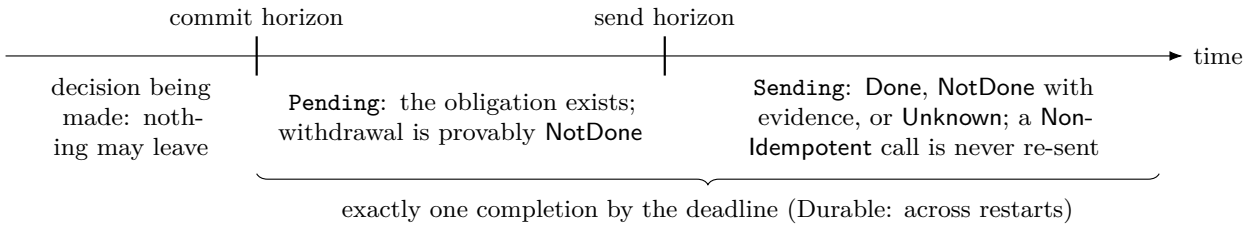



Figure 1: The two boundaries in time that every call crosses. Before the commit horizon there is nothing to know; between the horizons non-occurrence can still be proved; after the send horizon only the destination's answer or later reconciliation establishes what happened.

**Ports and effect classes.** A *port* is a typed request/response contract owned by its destination. Its *effect class* states what repeating a request does there:

- Safe: no effect;
- Idempotent: the destination deduplicates by the call's identity;
- NonIdempotent: every delivery may take effect.

The vocabulary is HTTP's [17]. The same distinction appears in hardware as Normal versus Device memory [1] and as posted versus non-posted requests [47].

**Obligations and outcomes.** A committed call is an *obligation*: the runtime must deliver exactly one completion for it to the caller's `complete`, by a finite deadline. The completion's outcome is one of the following:

- Done(value): the destination processed the request;
- NotDone(reason): there is evidence that the request had no effect;
- Unknown(reason): the request may or may not have taken effect.

**Profiles.** Under the *Transient* profile nothing survives a crash. Under the *Durable* profile the committed state survives, together with the call ledger and its phases, recently answered requests and unacknowledged notices.

## 3 Boundaries in Time

Spatial boundaries constrain which code may refer to which. Ballarch adds two boundaries that constrain *when* effects and knowledge may cross (Figure 1):

**Commit horizon.** The atomic commit of the decision that requests a call. Before it, the call is part of a decision that can still vanish, and nothing may leave (commit before dispatch). After it, the call is an obligation.

**Send horizon.** The first moment the request may have left the runtime. Before it, the call can be withdrawn truthfully. After it, NotDone requires evidence from the destination.

An implementation must make each horizon an event that its checks can observe:

- in the Kotlin runtime: a store write, then a write-ahead `Sending` record written before the transport is invoked;
- in firmware: the CRC-checked NVM record, then the write-ahead record before the UART transmission;

| Situation | Safe | Idempotent | NonIdempotent |
|---|---|---|---|
| Destination answered | Done | Done | Done |
| Deadline or cancel before the send horizon | NotDone | NotDone | NotDone |
| Evidence of non-application, one attempt | NotDone | NotDone | NotDone |
| No answer, attempts and time left | retry | retry | Unknown |
| No answer, no attempts or time left | NotDone | Unknown | Unknown |
| Durable restart after the send horizon | re-send or NotDone | re-send or Unknown | Unknown |
| Durable restart before the send horizon | send | send | send |

Table 1: The runtime's classification of a call's single outcome. A Safe call that gets no answer ends NotDone(NoAnswer), because it can have had no effect. A report that arrives after the call resolved is dropped.

- in a synchronous circuit: the clock edge, then the bus handshake.

The mechanisms behind both horizons are established:

- the output-commit rule of rollback recovery [15, 53];
- write-ahead logging [42];
- operating systems that hold external output until it no longer depends on uncommitted state [43, 44];
- in-order retirement in processors [52].

Real-time architectures specify component interfaces in the temporal domain [33], but for timing predictability rather than for knowledge about effects. To our knowledge, no software architecture style has named these instants as architectural boundaries and attached rules about outcomes to them.

## 4 Outcomes from Phase and Effect Class

Table 1 gives the classification that a conforming runtime applies. Two rules carry its weight:

**R7 (honest outcomes).** NotDone is produced only with evidence: the call was never sent, the destination refused it, or an adapter reported evidence covering every attempt that may have been delivered.

**R8 (no blind re-send).** A NonIdempotent call is sent at most once, including after a restart. Before each send, the runtime durably records that the call may have been sent.

Retries belong to the runtime and are allowed only for Safe and Idempotent ports. Adapters report each attempt once and never retry. Table 2 lists all fourteen rules with the mechanisms that check them.

The decider sees one completion per call and must handle all three outcomes. The type system makes Unknown a case, not an exception that each call site maps differently:

```
when (val outcome = completion.outcomeOf(Mailer)!!) {
    is Outcome.Done -> step(ReminderState.Sent)
    is Outcome.NotDone -> step(ReminderState.NotSent(outcome.reason.toString()))
    is Outcome.Unknown -> step(ReminderState.MaybeSent) // never re-send blindly
}
```

| Rule | Binds | Statement (abridged) | Checked in this work by |
|---|---|---|---|
| R1 Single owner | author | one owner per fact; immutable values | bytecode rules; C object-file check; Yosys single-driver check |
| R2 Pure decider | author | no I/O, clocks, randomness, globals | bytecode rule; C undefined-symbol check; Yosys latch check |
| R3 Serial processing | runtime | no concurrent or nested steps per instance | runtime guard; trace oracle |
| R4 Atomic commit | runtime | state and outputs in one atomic write | store design; Alloy P6; power-loss exploration |
| R5 Commit before dispatch | runtime | nothing leaves before its commit | Alloy P1; oracles; Yosys path check |
| R6 One completion per call | runtime | exactly one by the deadline | Alloy P2, L1; oracles |
| R7 Honest outcomes | runtime, adapter | NotDone only with evidence | Alloy P3, P5; oracles with ground truth |
| R8 No blind re-send | runtime, adapter | NonIdempotent sent at most once; write-ahead | Alloy P4; oracles; seeded bugs |
| R9 One reply per request | runtime, author | at most one reply; overdue replies reported | preflight; oracle |
| R10 Explicit composition | author | interaction only via calls, replies, notices | kernel API; module-boundary check |
| R11 Bounded resources | runtime | refuse before accepting | preflight |
| R12 Read-only queries | author | reads do not change state | types |
| R13 Honest profile claims | runtime | no claim beyond the profile | review |
| R14 Honest composition | author | claims about composites follow the algebra | `Knowledge` type; exploration against ground truth |

Table 2: The rules of Ballarch Core 2.0 and the mechanisms that check them in the three references.

# 5 Knowledge of Composite Operations

A Ball often performs one operation through several calls: a flight and a hotel, a reservation and a charge. Workflows usually report such operations as succeeded or failed. After a partial or uncertain effect, both labels can be false.

**Definition 1** (Truth values)**.** *Let* $\mathbb{T} = \{1, 0, \bot\}$ *with Kleene's strong connectives [32] and the information order* $\bot \sqsubseteq 1$, $\bot \sqsubseteq 0$. *A value* $t$ admits *a Boolean* $b$ *if* $t = \bot$ *or* $t = [b]$.

**Definition 2** (Constituents and knowledge)**.** *The* effect $f_i \in \mathbb{T}$ *of a call made for an operation is:*

- 1 *if it completed* Done *with an answer meaning that the requested effect happened;*
- 0 *if it completed* NotDone*, or was never sent;*
- $\bot$ *if it completed* Unknown*, or is still open.*

*A compensation is* absorbing *if its effect means that no effect of the original request exists or can still take effect. For example, the destination fences the request's reference, or the original request's delivery horizon has passed. With* $u_i$ *the disjunction of the effects of a constituent's absorbing compensations, its* net effect *is* $n_i = f_i \wedge \neg u_i$. *The* knowledge *of an operation with constituents* $1..k$ *is*

$$K = (all, any) = \Big( \bigwedge_i n_i,\ \bigvee_i n_i \Big), \qquad K_\emptyset = (1, 0).$$

Composition $(a_1, s_1) + (a_2, s_2) = (a_1 \wedge a_2,\ s_1 \vee s_2)$ is a commutative monoid with identity $K_\emptyset$, the knowledge of an operation that requested no effect. Seven values can occur:

- Done $(1, 1)$ and NotDone $(0, 0)$;
- Partial $(0, 1)$: some effects happened and some did not;
- NotAllDone $(0, \bot)$ and SomeDone $(\bot, 1)$;
- Unknown $(\bot, \bot)$;
- NoEffects $(1, 0)$.

Seen as one call, a composite is Done only if *all* is 1, NotDone only if *any* is 0, and Unknown otherwise.

**Theorem 1** (Soundness)**.** *If each net effect $n_i$ admits the true net effect $b_i$, then* all *admits $\bigwedge_i b_i$ and* any *admits $\bigvee_i b_i$.*

*Proof.* Kleene's strong connectives are monotone in $\sqsubseteq$ and agree with Boolean ones on definite values. Replacing each $\bot$ in $n$ by the corresponding $b_i$ refines $n$ to $b$, so $\bigwedge n_i \sqsubseteq \bigwedge b_i$, which is the claim. The same holds for $\bigvee$ and for $f_i \wedge \neg u_i$. □

**Theorem 2** (Coordinate-wise exactness)**.** *Let each $f_i$ and the effect of each compensation attempt be the most informative value admitting every possibility for that call alone, with calls independent. Then each coordinate of $K$ is the most informative value admitting every combination of true effects consistent with the calls' outcomes.*

*Proof. all*, *any* and $f_i \wedge \neg u_i$ are formulas in which every variable occurs once. On such formulas, strong Kleene evaluation coincides with supervaluation: the value is definite exactly when all completions agree. □

**Corollary 1** (Composition closure)**.** *A Ball that derives every claim about a composite operation from K (rule R14) makes, through its port, only claims that admit the truth. One level up, it is therefore a constituent whose knowledge is honest, and by induction the property holds at any depth of composition.*

The kernel's `Knowledge` type implements the algebra. An exhaustive test checks three things:

- the connectives against supervaluation;
- the monoid laws on all seven values;
- soundness and exactness for up to four constituents, each with or without a compensation: 69,905 worlds.

Associativity extends the result to any number of constituents.

Exactness is relative to the constituents' individual outcomes. If $K(e)$ reads "it is known that $e$", conjunction matches epistemic logic, since $K(a \wedge b) \Leftrightarrow Ka \wedge Kb$ [23], but disjunction does not. Evidence that correlates constituents, such as a log showing that one of two calls landed without saying which, can establish more than the algebra derives. The algebra under-approximates such knowledge and never over-approximates it.

**Example.** A trip Flow books a flight, which is NonIdempotent, and a hotel, which is Idempotent. It compensates with absorbing voids when it cannot hold both, and reports only $K$. The exploration ran 1,339 runs within a bound of twelve choices per run, covering every combination of five fates per booking attempt, three per compensation attempt, and one crash before any of the first ten scheduler tasks. In every run, its settled knowledge admitted the providers' real bookings. The resulting knowledge values were Done 2, NotDone 282, NotAllDone 620, Partial 64, SomeDone 92, and Unknown 276. On the same runs, a two-valued status ("booked" if both bookings answered booked,

| Concept | Service (Kotlin) | Firmware (C11) | Hardware (SystemVerilog) |
|---|---|---|---|
| Owner (R1) | runtime-held state | one NVM record | registers with one writing process |
| Decider (R2) | pure functions | `step()` in its own unit | one latch-free `always_comb` |
| Commit horizon | one store write | CRC-checked slot write | the clock edge |
| Send horizon | write-ahead `Sending` | write-ahead record | `valid && ready` (AXI: VALID) |
| Unknown | timeout or restart while `Sending` | power loss while `Sending` | accepted transaction without response |
| Profile | Transient or Durable | Durable | Transient |

Table 3: The same concepts at three levels.

otherwise "failed") was wrong in 254 runs. An absorbing compensation turns an Unknown booking into a definite NotDone, since $\bot \wedge \neg 1 = 0$. This is why compensation removes most ambiguity in practice.

## 6 One Rule Set at Every Level

The rules mention no language, transport or storage. Table 3 maps the concepts onto the three references; each reference also carries seeded bugs that its checks must catch.

**Services.** The Kotlin reference consists of the following parts:

- a dependency-free kernel of about 260 lines of code, including the knowledge algebra;
- a runtime of about 1,000 lines that implements both profiles, with an in-memory store for simulation and a file store that commits by atomic rename;
- a test kit whose oracles check R3–R9 on execution traces against simulated ground truth, under exhaustive or seeded exploration of network fates and crash points in the style of stateless model checking [20];
- architecture rules that check R1, R2 and R10 on bytecode.

**Firmware.** The C reference commits state and the call ledger as one CRC-checked record in two NVM slots. It records each send ahead of transmission, so a power loss after the write-ahead record turns a dose command into Unknown rather than a second dose. Its example is an infusion-pump controller whose pump has no idempotency key. An exhaustive exploration covers about 220,000 runs over every link fate and every power-loss point, including torn writes. No violation of R5–R8 occurred, nor any overdose or mismatch between the controller's belief and the pump. A check of the decider's object file enforces R2 by allowing no undefined symbol, and R1 by allowing no writable data symbol.

**Hardware.** The RTL reference is a command issuer:

- its registers have one writer;
- its next-state logic is one latch-free `always_comb` block;
- its outputs depend on registered state only.

It issues READ (Safe), CONFIG (Idempotent) and FIRE (NonIdempotent) transactions on a bus with id-tagged responses. Yosys checks single drivers, the absence of latches, and the absence of any combinational path from an input to a dispatched output. A fault-injecting bench runs 75 scenarios

against a device model. The first draft of the block reported NotDone when a CONFIG retry was withdrawn before acceptance, although the first attempt had already been applied; the bench caught it. Hardware engineers apply such rules by experience. Stating them as rules lets a tool check them.

**Closest work.** Bluespec's guarded atomic actions and the Bluespec Codesign Language compile one rule-based semantics to both hardware and software [29, 45]. Lingua Franca reactors and synchronous languages give deterministic semantics across targets [4, 22, 36]. These unify how computation executes. Ballarch unifies what is known about external effects and checks it at each level.

# 7 Formal Model of the Call Protocol

An Alloy 6 model [28] covers the protocol of Table 1:

- commit, write-ahead, send, retry, deadline, crash and recovery;
- a lossy, duplicating network;
- the three effect classes.

Within bounds of three calls and traces of up to 14 steps, it has no counterexample to six safety properties: no send before commit, stable outcomes, no false NotDone, at most one send of a NonIdempotent call, Done only after processing, and agreement of state and call ledger. Under fairness, with two calls and up to 12 steps, every awaited call gets an outcome. Each of six seeded runtime bugs yields a counterexample, for example sending before the write-ahead record or re-sending after a crash. These are bounded results, not proofs for all sizes.

# 8 Evaluation

We ask three questions:

**RQ1.** Under provider faults and crashes, do implementations that follow each style's canonical guidance meet the correctness requirements?

**RQ2.** What does Ballarch cost and gain structurally, compared with a design hardened by hand to the same correctness?

**RQ3.** Do the oracles detect seeded bugs in the correct implementations?

**Scenario.** Three orders compete for two units of stock. Each order reserves a unit, charges 100 through a provider *without idempotency keys*, and completes; on failure it releases the unit. The provider's status endpoint is Safe. A database with atomic transactions survives crashes. The requirements are:

- no double charge;
- no order reported failed while charged, and none reported completed while not charged;
- every accepted order ends in a terminal state;
- no stock leak, oversell or charge for an unaccepted order.

A NeedsAttention terminal state is allowed and is reported as a cost.

| Architecture | Stress | Stress + crash | Mild | Mild + crash |
|---|---|---|---|---|
| Layered (Clean Architecture) | 73.4 | 67.3 | 11.7 | 57.9 |
| Layered, hardened by hand | 0.0 | 0.0 | 0.0 | 0.0 |
| Actors with durable state | 73.4 | 68.2 | 11.7 | 35.9 |
| MVU (Elm architecture) | 73.4 | 50.7 | 11.7 | 48.5 |
| Hexagonal + outbox + saga | 73.4 | 72.5 | 11.7 | 24.9 |
| Ballarch | 0.0 | 0.0 | 0.0 | 0.0 |

Table 4: RQ1: percentage of runs (of 2,000 per cell) that violated at least one requirement. Wilson 95% intervals have a half-width of at most 2.2 points; for 0.0 the upper bound is 0.2. Without crashes, the four idiomatic designs behave identically: each maps a timeout or error to failure, with one attempt per order.

**Fault model.** Each provider request draws one fate: answered in time, answered late, lost, processed with the answer lost, processed then reset, or refused before sending.

- *Stress* draws all six fates with equal probability.
- *Mild* answers 90% of requests in time.

In crash cells, the process crashes once, before a scheduler task chosen uniformly within a typical run of that architecture, and restarts 50 ms later. The simulation is deterministic; rerunning it reproduces the published data byte for byte.

**Implementations.** The same author wrote all six against this specification. Code that stands in for a framework is excluded from the structural metrics:

**Layered** Clean Architecture [39]. One attempt with a timeout; any exception fails the order. On restart it resumes unfinished orders.

**Hardened** The same layers, plus a hand-written write-ahead status, retries only after a refused connection, and reconciliation by status query.

**Actors** Durable state and `ask` with a timeout. On restart, actors are re-driven.

**MVU** The Elm architecture [14] on the server. The model is persisted before commands run, and `init` re-issues them.

**Saga** Hexagonal architecture with a transactional outbox, an orchestrated saga and idempotent consumers [50].

**Ballarch** Inventory, payment and checkout Balls on the reference runtime.

**RQ1.** Table 4 shows the results. The violations of the idiomatic designs come from two sources:

- a timeout or reset after the charge took effect, reported as a failure (false failures);
- recovery paths that re-send a charge that may already have happened (double charges): resumed layered processing, re-driven actors, re-issued MVU commands, and outbox redelivery to a consumer whose provider call lies outside its transaction.

In the Mild-with-crash setting, for example, the layered design double-charged in 1,110 runs and the saga in 338. Actors also leaked stock when a crash lost a fire-and-forget release. Ballarch and the hardened design had no violation in any of the 16,000 runs between them. The cost of their honesty is NeedsAttention: 0.27–0.31 per order under Stress, and at most 0.001 under Mild.

**RQ2.** Table 5 compares the structures. Against the hardened design, which is equally correct, Ballarch has these costs:

- 29% more source lines and 28% more bytecode;

| Metric | Layered | Hardened | Actors | MVU | Saga | Ballarch |
|---|---|---|---|---|---|---|
| SLOC | 166 | 216 | 163 | 96 | 184 | 279 |
| Classes | 12 | 20 | 15 | 10 | 18 | 35 |
| CBO, mean | 4.00 | 4.70 | 3.47 | 4.00 | 4.11 | 3.94 |
| LCOM4, share = 1 | 71% | 71% | 67% | 50% | 67% | 8% |
| CAMC, mean | 0.52 | 0.52 | 0.39 | 0.42 | 0.57 | 0.33 |
| Distance $D$, mean | 0.34 | 0.35 | 0.42 | 0.39 | 0.47 | 0.44 |
| Propagation cost | 28.5% | 20.0% | 21.8% | 31.0% | 20.7% | 13.1% |
| McCabe CCN, mean (max) | 2.05 (8) | 1.81 (7) | 1.60 (4) | 1.75 (7) | 2.46 (13) | 2.19 (8) |
| Framework types | 5 | 6 | 5 | 3 | 6 | 45 |
| `catch` clauses | 3 | 5 | 0 | 0 | 2 | 0 |
| Code in pure classes | 26% | 27% | 26% | 59% | 44% | 88% |

Table 5: RQ2: structure of the feature code. Metrics: CBO [12], LCOM4 [27], CAMC [3], distance from the main sequence [38], propagation cost [37], and McCabe complexity [40] via lizard [57]. Structural metrics come from bytecode; compiler-generated classes are merged into their declaring class.

- 45 framework types against 6;
- more, smaller classes;
- lower cohesion.

LCOM4 assumes that cohesive methods share fields. It counts stateless deciders over a state type as uncohesive, which affects MVU's `update` as well. The type-based CAMC metric does not reverse the result. In exchange, Ballarch has the lowest propagation cost of the six designs. It also places 88% of its code in classes with no mutable field and no reference to concurrency, I/O, clocks, the database or the provider, against 27% for the hardened design. Its feature code has no exception handler. The hardened design reaches the same correctness through five `catch` clauses, a write-ahead status and reconciliation, all written into the use case.

**RQ3.** We injected two plausible bugs from outside the correct implementations: a Ballarch payment Ball that treats `Unknown` as not charged, and a hardened gateway that reports an uncertain charge as not sent. Under Stress with a crash they were detected in 79.4% and 46.4% of runs respectively, and under Mild with a crash in 47.3% and 2.3%.

# 9 Related Work

**Architecture styles.** Architecture research describes styles as components, connectors and constraints [48, 51], and modules by the decisions they hide [46].

- Layered, Clean and Hexagonal architectures and domain-driven design [13, 16, 39] constrain dependencies.
- Actor systems give each piece of state one owner [2, 10, 26].
- The Elm architecture and the Decider pattern separate pure transitions from effects [11, 14].
- Durable-execution systems provide deterministic replay and exactly-once semantics for orchestrations [8].

Ballarch keeps the structural constraints of these styles. To them it adds the temporal boundaries, the outcome rules and the algebra. Meng and Jackson's structural pattern for legible software is a recent example of a new structural pattern evaluated on an application [41].

**Ambiguous outcomes and idempotence.** Several lines of work recognise ambiguous outcomes:

- the "maybe" outcome of RPC [5];
- Jepsen's indeterminate operations [30, 31];
- the heuristic outcomes of XA [56], including mixed and hazard;
- Helland's analysis of activities between partners [24, 25].

X-ability defines exactly-once correctness for services with side effects and is composable [18]. Ramalingam and Vaswani make workflows idempotent and therefore safe to retry [49]. These approaches hide or remove the third outcome. Ballarch surfaces it, chooses it mechanically, and composes it. Recent studies of LLM agents report duplicated and hidden side effects at tool boundaries [35, 55], a new setting for the same problem.

**Compensation and three-valued reasoning.** Sagas [19] and compensation calculi [6, 9] give semantics to long-running transactions, but assume that outcomes are known. Three-valued abstractions and may/must approximations [7, 34] use the monotonicity argument of our soundness theorem. Knowledge-based analyses of commitment [21, 23] reason about what processes know. The algebra here is truth-functional by design, so that a runtime can compute it.

**Output commit and hardware.** The output-commit problem [15, 53], write-ahead logging [42], external synchrony [43, 44] and precise interrupts [52] are the mechanisms behind the horizons. The time-triggered architecture [33] makes time part of interfaces. Bluespec, BCL, Lingua Franca and synchronous languages [4, 22, 29, 36, 45] unify execution semantics across hardware and software.

## 10 Limitations and Threats to Validity

- **Construct and internal validity.** One author wrote all implementations and designed the architecture. Each idiomatic baseline is one reading of its style's guidance; the hardened baseline shows that an expert can reach correctness without Ballarch. Independent implementations, written before seeing the results, would strengthen the comparison.
- **External validity.**
  - The scenario is small and uses a simulated provider without idempotency keys; with idempotency keys, simple retries would become correct.
  - The fault distributions are assumptions.
  - The firmware and RTL references are single examples checked in simulation.
  - The Kotlin runtime's file store is single-node and has been tested against killed processes, not power loss; it is not a production database.
- **Metrics.** LCOM4 and CAMC were designed for object-oriented code with shared fields. Propagation cost and CBO depend on how finely code is divided into classes. Change impact, developer effort and runtime overhead were not measured in this study. An earlier, unpublished prototype comparison found wider changes for Ballarch's module structure and a median of 39 µs of thread hand-off per local request.
- **Formal results.** The Alloy results are bounded. The algebra's theorems are proved, and are also checked exhaustively for small sizes. The absorbing property of compensations and the honesty of adapters are assumptions that are reviewed, not verified.

## 11 Conclusion

Ballarch places two instants that mechanisms in recovery, operating systems and processors already respect into the architecture itself, as boundaries in time. A runtime derives each outcome from them, and a small algebra carries honesty from parts to wholes. The same rules can be checked from register-transfer logic to services. In our benchmark, correctness under faults required either Ballarch or an expert's hand-written fault handling. Ballarch moved that knowledge from the feature into the architecture, at the price of more code and more concepts. Whether that trade pays off for teams, and for coding agents that write such features, is the next question to measure.

**Artifacts.** The specification, the Kotlin, C and SystemVerilog references, the Alloy model, the benchmark and its raw data are available at `https://github.com/4wl2d/Pokeball` (version 2.0.0-rc.1). The file `docs/problems.md` there maps twelve recurring failures of current architectures to the mechanism, rule and evidence for each, and lists what Ballarch does not solve. Text is licensed under CC BY 4.0; the code has no software license yet.

**Use of generative AI.** The author used an AI assistant (Anthropic's Claude) to draft parts of the code, the benchmark and the text, under the author's direction. The author reviewed the material, ran the reported checks, and takes full responsibility for the content.

## References


[1] Arm Limited. *Arm Architecture Reference Manual for A-profile Architecture.* Arm Limited, issue l.b edition, 2025. URL `https://developer.arm.com/documentation/ddi0487/latest`.

[2] Joe Armstrong. *Making Reliable Distributed Systems in the Presence of Software Errors.* PhD thesis, The Royal Institute of Technology (KTH), Stockholm, Sweden, 2003.

[3] Jagdish Bansiya, Letha Etzkorn, Carl Davis, and Wei Li. A class cohesion metric for object-oriented designs. *Journal of Object-Oriented Programming*, 11(8):47–52, 1999.

[4] Gérard Berry and Georges Gonthier. The Esterel synchronous programming language: Design, semantics, implementation. *Science of Computer Programming*, 19(2):87–152, 1992. doi: 10.1016/0167-6423(92)90005-V.

[5] Andrew D. Birrell and Bruce Jay Nelson. Implementing remote procedure calls. *ACM Transactions on Computer Systems*, 2(1):39–59, 1984. doi: 10.1145/2080.357392.

[6] Roberto Bruni, Hernán C. Melgratti, and Ugo Montanari. Theoretical foundations for compensations in flow composition languages. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '05)*, pages 209–220. ACM, 2005. doi: 10.1145/1040305.1040323.

[7] Glenn Bruns and Patrice Godefroid. Model checking partial state spaces with 3-valued temporal logics. In *Computer Aided Verification (CAV '99)*, volume 1633 of *Lecture Notes in Computer Science*, pages 274–287. Springer, 1999. doi: 10.1007/3-540-48683-6_25.

[8] Sebastian Burckhardt, Chris Gillum, David Justo, Konstantinos Kallas, Connor McMahon, and Christopher S. Meiklejohn. Durable functions: Semantics for stateful serverless. *Proceedings of the ACM on Programming Languages*, 5(OOPSLA):133:1–133:27, 2021. doi: 10.1145/3485510.

[9] Michael Butler, C. A. R. Hoare, and Carla Ferreira. A trace semantics for long-running transactions. In *Communicating Sequential Processes: The First 25 Years*, volume 3525 of *Lecture Notes in Computer Science*, pages 133–150. Springer, 2005. doi: 10.1007/11423348_8.

[10] Sergey Bykov, Alan Geller, Gabriel Kliot, James R. Larus, Ravi Pandya, and Jorgen Thelin. Orleans: Cloud computing for everyone. In *Proceedings of the 2nd ACM Symposium on Cloud Computing (SoCC '11)*. ACM, 2011. doi: 10.1145/2038916.2038932.

[11] Jérémie Chassaing. Functional event sourcing decider. `https://thinkbeforecoding.com/post/2021/12/17/functional-event-sourcing-decider`, December 2021. Blog post, Accessed 2026-10-21.

[12] Shyam R. Chidamber and Chris F. Kemerer. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493, 1994. doi: 10.1109/32.295895.

[13] Alistair Cockburn. Hexagonal architecture. `https://alistair.cockburn.us/hexagonal-architecture/`, 2005. Accessed 2026-10-21.

[14] Evan Czaplicki. Elm: Concurrent FRP for functional GUIs. Senior thesis, Harvard University, 2012.

[15] E. N. Elnozahy, Lorenzo Alvisi, Yi-Min Wang, and David B. Johnson. A survey of rollback-recovery protocols in message-passing systems. *ACM Computing Surveys*, 34(3):375–408, 2002. doi: 10.1145/568522.568525.

[16] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003. ISBN 0-321-12521-5.

[17] Roy T. Fielding, Mark Nottingham, and Julian Reschke. HTTP semantics. RFC 9110, STD 97, June 2022. URL `https://www.rfc-editor.org/info/rfc9110`.

[18] Svend Frølund and Rachid Guerraoui. X-ability: A theory of replication. *Distributed Computing*, 14(4):231–249, 2001. doi: 10.1007/s004460100065.

[19] Hector Garcia-Molina and Kenneth Salem. Sagas. In Umeshwar Dayal and Irving L. Traiger, editors, *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data, San Francisco, CA, USA, May 27–29, 1987*, pages 249–259. ACM Press, 1987. doi: 10.1145/38713.38742.

[20] Patrice Godefroid. Model checking for programming languages using VeriSoft. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '97)*, pages 174–186. ACM, 1997. doi: 10.1145/263699.263717.

[21] Vassos Hadzilacos. A knowledge theoretic analysis of atomic commitment protocols. In *Proceedings of the Sixth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems (PODS '87)*, pages 129–134. ACM, 1987. doi: 10.1145/28659.28672.

[22] Nicolas Halbwachs, Paul Caspi, Pascal Raymond, and Daniel Pilaud. The synchronous data flow programming language LUSTRE. *Proceedings of the IEEE*, 79(9):1305–1320, 1991. doi: 10.1109/5.97300.

[23] Joseph Y. Halpern and Yoram Moses. Knowledge and common knowledge in a distributed environment. *Journal of the ACM*, 37(3):549–587, 1990. doi: 10.1145/79147.79161.

[24] Pat Helland. Life beyond distributed transactions: an apostate's opinion. In *Third Biennial Conference on Innovative Data Systems Research (CIDR 2007), Asilomar, CA, USA, January 7–10, 2007, Online Proceedings*, pages 132–141. www.cidrdb.org, 2007. URL `http://cidrdb.org/cidr2007/papers/cidr07p15.pdf`.

[25] Pat Helland and David Campbell. Building on quicksand. In *Fourth Biennial Conference on Innovative Data Systems Research (CIDR 2009), Asilomar, CA, USA, January 4–7, 2009, Online Proceedings*. www.cidrdb.org, 2009.

[26] Carl Hewitt, Peter Boehler Bishop, and Richard Steiger. A universal modular ACTOR formalism for artificial intelligence. In Nils J. Nilsson, editor, *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI), Stanford, CA, USA, August 20–23, 1973*, pages 235–245. William Kaufmann, 1973. URL `https://www.ijcai.org/Proceedings/73/Papers/027B.pdf`.

[27] Martin Hitz and Behzad Montazeri. Measuring coupling and cohesion in object-oriented systems. In *Proceedings of the International Symposium on Applied Corporate Computing (ISACC '95)*, Monterrey, Mexico, 1995.

[28] Daniel Jackson. *Software Abstractions: Logic, Language, and Analysis*. MIT Press, Cambridge, MA, revised edition, 2012. ISBN 978-0-262-01715-2.

[29] Myron King, Nirav Dave, and Arvind. Automatic generation of hardware/software interfaces. In *Proceedings of the 17th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XVII)*, pages 325–336. ACM, 2012. doi: 10.1145/2150976.2151011.

[30] Kyle Kingsbury. Jepsen: Distributed systems safety research. `https://jepsen.io`, 2026. Accessed 2026-10-21.

[31] Kyle Kingsbury and Peter Alvaro. Elle: Inferring isolation anomalies from experimental observations. *Proceedings of the VLDB Endowment*, 14(3):268–280, 2020. doi: 10.14778/3430915.3430918.

[32] Stephen Cole Kleene. *Introduction to Metamathematics*. North-Holland, Amsterdam, 1952. Co-published with P. Noordhoff (Groningen) and D. Van Nostrand (New York).

[33] Hermann Kopetz and Günther Bauer. The time-triggered architecture. *Proceedings of the IEEE*, 91(1):112–126, 2003.

[34] Kim Guldstrand Larsen and Bent Thomsen. A modal process logic. In *Proceedings of the Third Annual Symposium on Logic in Computer Science (LICS '88)*, pages 203–210. IEEE Computer Society, 1988. doi: 10.1109/LICS.1988.5119.

[35] Jiapeng Li. Where does exactly-once live? model, harness, and tool-contract effects on duplicate side effects in LLM agents, 2026.

[36] Marten Lohstroh, Christian Menard, Soroush Bateni, and Edward A. Lee. Toward a lingua franca for deterministic concurrent systems. *ACM Transactions on Embedded Computing Systems*, 20(4):36:1–36:27, 2021. doi: 10.1145/3448128.

[37] Alan MacCormack, John Rusnak, and Carliss Y. Baldwin. Exploring the structure of complex software designs: An empirical study of open source and proprietary code. *Management Science*, 52(7):1015–1030, 2006. doi: 10.1287/mnsc.1060.0552.

[38] Robert C. Martin. OO design quality metrics: An analysis of dependencies. White paper, October 1994. Dated October 28, 1994.

[39] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Robert C. Martin Series. Prentice Hall, 2017. ISBN 978-0-13-449416-6.

[40] Thomas J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, SE-2 (4):308–320, 1976. doi: 10.1109/TSE.1976.233837.

[41] Eagon Meng and Daniel Jackson. What you see is what it does: A structural pattern for legible software. In *Proceedings of the 2025 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! '25)*, pages 178–193. ACM, 2025. doi: 10.1145/3759429.3762628.

[42] C. Mohan, Donald J. Haderle, Bruce G. Lindsay, Hamid Pirahesh, and Peter M. Schwarz. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992. doi: 10.1145/128765.128770.

[43] Edmund B. Nightingale, Peter M. Chen, and Jason Flinn. Speculative execution in a distributed file system. In *Proceedings of the 20th ACM Symposium on Operating Systems Principles (SOSP '05)*, pages 191–205. ACM, 2005. doi: 10.1145/1095810.1095829.

[44] Edmund B. Nightingale, Kaushik Veeraraghavan, Peter M. Chen, and Jason Flinn. Rethink the sync. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06)*, pages 1–14. USENIX Association, 2006.

[45] Rishiyur S. Nikhil. Bluespec system verilog: Efficient, correct RTL from high level specifications. In *Proceedings of the 2nd ACM/IEEE International Conference on Formal Methods and Models for Co-Design (MEMOCODE 2004)*, pages 69–70. IEEE, 2004. doi: 10.1109/MEMCOD.2004.1459818.

[46] David Lorge Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972. doi: 10.1145/361598.361623.

[47] PCI-SIG. *PCI Express Base Specification, Revision 7.0*. PCI-SIG, 2025.

[48] Dewayne E. Perry and Alexander L. Wolf. Foundations for the study of software architecture. *ACM SIGSOFT Software Engineering Notes*, 17(4):40–52, 1992. doi: 10.1145/141874.141884.

[49] Ganesan Ramalingam and Kapil Vaswani. Fault tolerance via idempotence. In *Proceedings of the 40th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '13)*, pages 249–262. ACM, 2013. doi: 10.1145/2429069.2429100.

[50] Chris Richardson. *Microservices Patterns: With Examples in Java*. Manning, 2018. ISBN 978-1-61729-454-9.

[51] Mary Shaw and David Garlan. *Software Architecture: Perspectives on an Emerging Discipline*. Prentice Hall, Upper Saddle River, NJ, 1996. ISBN 0-13-182957-2.

[52] James E. Smith and Andrew R. Pleszkun. Implementation of precise interrupts in pipelined processors. In *Proceedings of the 12th Annual Symposium on Computer Architecture (ISCA '85), Boston, MA, USA, June 1985*, pages 36–44. IEEE Computer Society, 1985. doi: 10.1145/327010.327125.

[53] Robert E. Strom and Shaula Yemini. Optimistic recovery in distributed systems. *ACM Transactions on Computer Systems*, 3(3):204–226, 1985. doi: 10.1145/3959.3962.

[54] Vladislav Tomilov. Ballarch: Core 2.0 specification, reference implementations and benchmark. `https://github.com/4wl2d/Pokeball`, 2026. Version 2.0.0-rc.1.

[55] Artem Trofimov and Boris Novikov. When tool calls succeed but workflows fail: Anomalies at the agent-tool boundary, 2026.

[56] X/Open Company Ltd. *Distributed Transaction Processing: The XA Specification*. X/Open, 1991.

[57] Terry Yin. Lizard: An extensible cyclomatic complexity analyzer. `https://github.com/terryyin/lizard`, 2026. Version 1.24.1, accessed 2026-10-21.